Robotics and Autonomous Systems 207 (2027) 105798

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

Infrastructure-based autonomous mobile robots for internal logistics — challenges and future perspectives

Erik Brorsson [a,b,*], Kristian Ceder [a], Ze Zhang [a], Sabino Francesco Roselli [a], Endre Erős [c], Martin Dahl [a], Beatrice Alenljung [d], Jessica Lindblom [e], Thanh Bui [f], Atieh Hanna [b], Emmanuel Dean [a], Lennart Svensson [a], Kristofer Bengtsson [b], Per-Lage Götvall [b], Knut Åkesson [a]

[a] Department of Electrical Engineering, Chalmers University of Technology, Gothenburg, Sweden
[b] Global Trucks Operations, Volvo Group, Gothenburg, Sweden
[c] Chalmers Industriteknik, Gothenburg, Sweden
[d] School of Informatics, University of Skövde, Skövde, Sweden
[e] Department of Information Technology, Uppsala University, Uppsala, Sweden
[f] RISE Research Institutes of Sweden, Gothenburg, Sweden

ARTICLE INFO

Keywords:
Autonomous mobile robots
Internal logistics
Cloud robotics
Infrastructure sensing
Multi-robot systems
Humanrobot interaction

ABSTRACT

The adoption of Autonomous Mobile Robots (AMRs) for internal logistics is accelerating, with most solutions emphasizing decentralized, onboard intelligence. While AMRs in indoor environments like factories can be supported by infrastructure, involving external sensors and computational resources, such systems remain underexplored in the literature. This paper presents a comprehensive overview of infrastructure-based AMR systems, outlining key opportunities and challenges. To support this, we introduce a reference architecture combining infrastructure-based sensing, on-premise cloud computing, and onboard autonomy. Based on the architecture, we review core technologies for localization, perception, and planning. We demonstrate the approach in a real-world deployment in a heavy-vehicle manufacturing environment and summarize findings from a user experience (UX) evaluation. Our aim is to provide a holistic foundation for future development of scalable, robust, and human-compatible AMR systems in complex industrial environments.

1. Introduction

The demand for automation in internal logistics is steadily increasing across a wide range of industrial sectors. While significant progress has been made in structured settings, such as fully automated warehouses, achieving effective automation in less regulated and dynamic environments remains a considerable challenge. For example, at Volvo Trucks' final assembly plants, the production process must accommodate a broad range of vehicle types, including battery-electric, internal combustion engine, and hydrogen-powered trucks. These variants are all assembled on the same production line, which significantly increases system complexity and requires a high degree of operational flexibility. This variability complicates automation efforts and reinforces the industry's reliance on human workers. Nevertheless, as automation technologies continue to mature, they are progressively introduced to complement the human workforce. In these environments, we can expect that humans and robots will work side by side, or in a collaborative sense, for the foreseeable future.

The need for flexibility in material handling has driven the development of Autonomous Mobile Robots (AMRs) [1]. AMRs are industrial, typically wheeled, robots that transport materials in dynamic, human-shared environments through decentralized decision-making and onboard navigation. Current solutions rely on onboard sensors, such as LiDAR and cameras, to perform simultaneous localization and mapping (SLAM) and detect obstacles in real-time for collision-free motion. Although these systems have seen substantial progress in recent years [1–8], several challenges remain. Generally, onboard sensors' limited range and susceptibility to occlusion hinder reliable perception in crowded environments and far away from the robot. SLAM is further complicated by moving objects that interfere with the mapping process.

* Corresponding author at: Department of Electrical Engineering, Chalmers University of Technology, Gothenburg, Sweden.
E-mail address: erikbro@chalmers.se (E. Brorsson).

https://doi.org/10.1016/j.robot.2026.105798
Received 11 May 2026; Received in revised form 17 July 2026; Accepted 3 October 2026
Available online 9 October 2026
0921-8890/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).



Fig. 1. Three of the autonomous transport robots operating inside the factory.

Additionally, mobile platforms are constrained by limited computational resources, which prohibits the use of computationally intensive algorithms for perception and decision making. These issues severely limit the robustness and reliability of AMR systems in complex indoors environments.

To overcome these limitations, recent research has proposed AMR systems that leverage cloud computing and infrastructure-based sensing under paradigms such as Cloud Robotics [9] and Internet of Robotic Things (IoRT) [10]. This offers multiple benefits, such as enhanced perception capabilities thanks to additional sensors and joint data processing, increased computational budget at the cloud or edge-clusters, and global coordination across a fleet of robots. Notable examples include: offloading algorithms for perception and planning to the cloud [11,12], multi-robot collaborative mapping [13] or SLAM [14–17], and the use of infrastructure-mounted sensors for localization [18,19], perception [19,20], or navigation [21–23].

However, the integration of infrastructure and cloud resources into AMR systems is still an emerging area of research. Recent surveys continue to focus primarily on onboard autonomy, and existing work involving infrastructure tends to address isolated functions or application-specific setups. A cohesive systems-level perspective is currently lacking. To help address this gap, we first propose a reference architecture for AMR systems that incorporates infrastructure-mounted sensors, cloud computing, and onboard intelligence to support a wide range of applications. Moreover, while key AMR capabilities such as localization, perception, and planning have been extensively studied in onboard-only systems, their design and performance are fundamentally different in infrastructure-supported setups. We therefore review these core functions from the perspective of the proposed architecture to identify emerging research opportunities and design considerations.

To ground our discussion, we also present a real-world industrial deployment of an infrastructure-based AMR system (see Fig. 1) and summarize insights from its evaluation. While prior work often remains conceptual or limited to lab settings, our implementation in a live factory environment highlights practical benefits and challenges. Finally, drawing from both literature and industrial experience, we identify critical open challenges and suggest promising directions for future research.

Our contributions can be summarized as follows:

- We propose a reference architecture for AMR systems that integrates infrastructure sensors and on-premise cloud computing resources.
- We review enabling technologies for key AMR functions, including localization, perception, and planning.
- We present an industrial deployment of the system and summarize key user experience (UX) evaluation results.
- Based on our review and evaluation, we identify current challenges and outline promising research directions.

The remainder of this paper is organized as follows. Section 2 defines the problem and outlines the reference architecture. Section 3 reviews essential supporting technologies. Section 4 presents our industrial evaluation. Section 5 discusses open research challenges and future directions. Finally, Section 6 concludes the paper.

2. Automated transports for internal logistics

2.1. Problem formulation

We address the problem of autonomously managing transportation tasks in a factory using a fleet of wheeled autonomous mobile robots (AMRs). The task set is defined as $\mathcal{T} = \{\tau_i\}_i$, where a task $\tau_i = [A, B, T_A, T_B]$ consists of a start location A, an end location B, and associated time windows T_A and T_B. Specifically, $T_A = [t_{A_0}, t_{A_1}]$ defines the permissible arrival window at location A, while $T_B = [t_{B_0}, t_{B_1}]$ defines the corresponding window at location B. The factory environment is dynamic due to uncontrollable events such as pedestrian movement and manually operated forklifts. At the same time, it is regulated by factory-specific traffic rules and conventions. Some are explicitly marked, for example with floor signage, while others are implicit through established practice. These rules define where robots may drive and which vehicle has priority at intersections. The objective is to control the AMR fleet to safely and efficiently complete the transportation tasks while complying with these rules.

Achieving this objective requires a system capable of perceiving and understanding its environment. First, the system must estimate the current state, including the locations of static and dynamic obstacles and the positions of all AMRs. As sensor data is noisy and incomplete, this estimate is inherently uncertain. To enable proactive behavior, the system must also predict likely future events, such as pedestrian or vehicle movements. These predictions are likewise uncertain and may change rapidly with human behavior. Developing an adequate understanding of the current and future state of the environment is therefore a key perception challenge that requires robust processing and interpretation of sensor data.

Beyond perception, the system must plan the actions of all AMRs. At the highest level, task assignment, scheduling, and route selection grows rapidly in complexity with the number of tasks, robots and routing alternatives, which makes it difficult to solve for larger instances. During execution, each robot must continuously select safe and effective actions to progress toward its goals while navigating near pedestrians and other vehicles. In crowded indoor settings, this involves reasoning about traffic rules and human intentions and may also require cooperation to avoid deadlocks. For example, some vehicles, such as factory tugger trains, cannot reverse, which may require strategic maneuvering from nearby AMRs. In summary, the system must respond quickly to environmental changes while maintaining predictable behavior that supports safe and effective human–robot collaboration, which pose a significant planning challenge.

2.2. Reference architecture

A common approach to autonomous navigation in dynamic environments is a modular software architecture that separates perception and decision-making components [4,7,8,24–27]. Perception is typically divided into localization, which estimates the robot's position, and environment perception. The latter includes estimating the current environment state (e.g., road layout and obstacle positions), tracking dynamic objects such as other road users, and predicting their future trajectories [27]. In our architecture, which is illustrated in Fig. 2, we summarize the perception module by localization, detection, object tracking, and motion prediction.

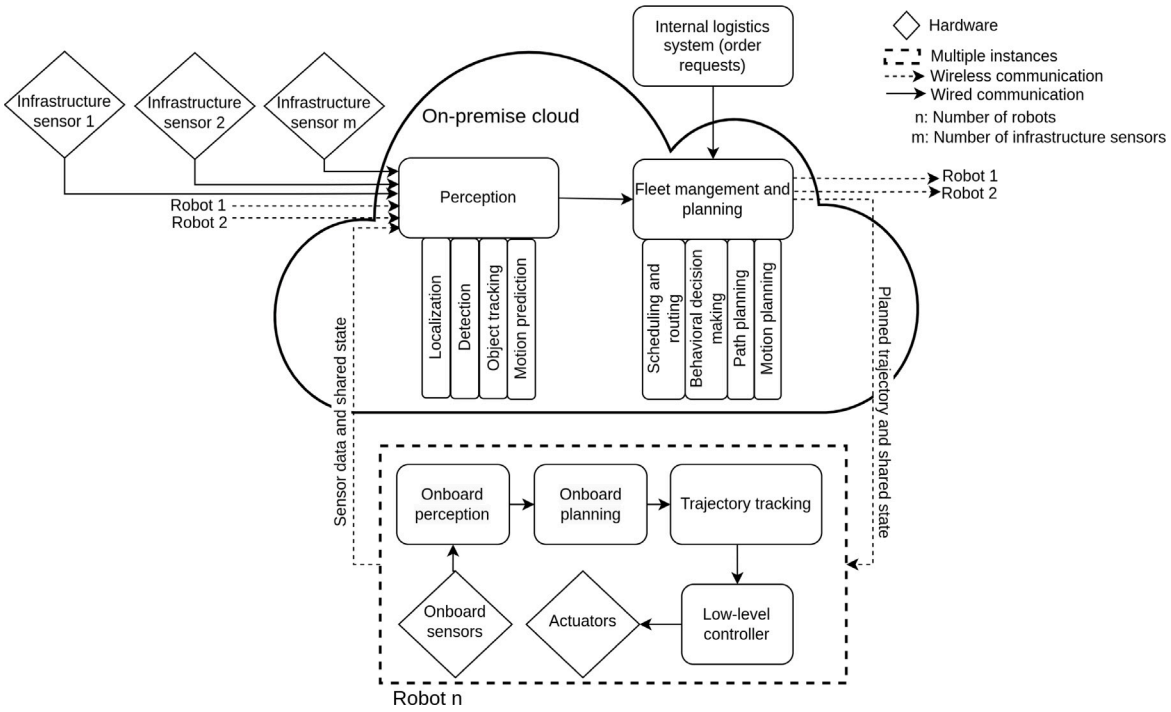


Fig. 2. RAIL: *a **R**eference **A**rchitecture for **I**nfrastructure-based AMR Systems in Internal **L**ogistics.*

Based on the perception results, the decision-making module provides a long-term route to the destination, makes short-to-mid-term decisions to progress toward it, and compute a feasible trajectory to execute the chosen behavior. In self-driving cars, this pipeline may be described by route planning, path planning, behavior selection, and motion planning [25]. By adding a task scheduler that allocates transportation tasks across the AMR fleet, the same structure applies to our application. Accordingly, we describe the decision making system as scheduling and routing, behavioral decision making, path planning, and motion planning in Fig. 2. The functionality of each perception and decision-making component is detailed in the next section.

Traditionally, both perception and decision-making have been performed entirely onboard, relying on sensors such as cameras, LiDAR, and radar. However, a growing trend in both autonomous vehicles and robotics is to leverage infrastructure and increase connectivity between agents. In autonomous driving, for example, vehicle-to-everything (V2X) communication is increasingly used for shared perception and cooperative planning [28]. Similarly, the robotics community has embraced cloud-based approaches in which key capabilities are offloaded from the robot to external infrastructure. For example, [12] demonstrates a fully cloud-offloaded navigation stack, where onboard sensor data is transmitted to a 5G-enabled edge cloud for real-time perception and control, with resulting commands sent back to the AMR. Other systems [14–17] focus on cloud-based SLAM execution with multiple robots. Infrastructure-mounted cameras have also been used for localization and guidance [21–23], with perception and planning executed directly on the camera hardware. Ceiling-mounted cameras further serve as auxiliary sensors to improve localization [19] and perception [19,20].

To leverage the benefits of infrastructure-supported autonomy, we propose RAIL: *a **R**eference **A**rchitecture for **I**nfrastructure-based AMR Systems in Internal **L**ogistics*. As illustrated in Fig. 2, RAIL generalizes and extends conventional AMR systems by incorporating infrastructure-mounted sensors, on-premise cloud computing, and onboard autonomy into a unified framework. This enables capabilities such as collaborative perception between robots, enhanced sensing through infrastructure devices, and centralized fleet coordination while preserving the flexibility to execute functions locally on the robot when appropriate.

In RAIL, AMRs and external sensors (e.g., ceiling-mounted cameras) transmit sensor data or extracted features to the on-premise cloud, which serves as a central hub for global perception and fleet coordination. Based on transportation requests from the internal logistics system, the cloud platform computes task schedules and motion plans in the form of trajectories, which are communicated wirelessly to the fleet. In addition, information is continuously exchanged between the robots and the cloud through a shared state, which represents the current knowledge of the system.

The shared state acts as a common information layer that enables collaboration between robots and infrastructure. Depending on the application, it may include robot states (e.g., position, velocity, and operational status), maps of the environment, occupancy representations, tracked dynamic objects and their predicted trajectories, transportation task status, planned routes and trajectories, and semantic information such as traffic rules or restricted areas. The exact content of the shared state is application dependent and may vary depending on communication bandwidth, computational resources, and system requirements. Consequently, the shared state should be regarded as an abstraction rather than a fixed data structure.

Although RAIL is designed to exploit infrastructure-based sensing and cloud computing, it does not require that all functionality is executed remotely. Instead, each functional component in Fig. 2 may execute onboard the robot, in the on-premise cloud, or be shared between them depending on the application requirements. For example, perception may rely entirely on infrastructure-mounted sensors, entirely on onboard sensors, or combine both through sensor fusion. Likewise, fleet scheduling and route planning may execute centrally while local collision avoidance or trajectory tracking remains onboard.

This flexibility enables several operating modes. During normal operation, computationally demanding tasks such as global perception, multi-robot coordination, and fleet optimization can be executed in the cloud, allowing robots to remain computationally lightweight. If communication is degraded or parts of the infrastructure become unavailable, robots may instead rely on onboard sensing and navigation to continue operating safely, although with reduced global awareness and coordination. Conversely, communication constraints may motivate partial task offloading, where robots process raw sensor data locally

Table 1
Qualitative comparison of the architectural trade-offs between conventional onboard AMR systems and the proposed RAIL reference architecture.

Architectural aspect	Conventional onboard systems	RAIL
Perception	Primarily based on onboard sensors with a local view of the environment.	Infrastructure sensors, onboard sensors, or a combination of both through collaborative perception.
Computation	Dedicated onboard processors execute perception and planning.	Computational tasks may execute onboard, in the on-premise cloud, or be shared between them.
Fleet management	Predominantly decentralized with global information limited to scheduling and routing.	Centralization across the entire stack enabling optimized scheduling, decision making, and path and motion planning.
Scalability	Each additional robot requires its own sensing and computational resources.	Infrastructure sensors and cloud resources are shared across the fleet, reducing the marginal cost of adding robots.
Robot cost	Higher sensing and computing cost per robot.	Lower per-robot hardware requirements, but higher initial infrastructure investment.
Adaptability	Decisions rely primarily on local observations.	Global perception enables proactive rerouting, congestion management, and rapid reconfiguration of transportation tasks.
Resilience	Naturally tolerant to infrastructure failures but limited by onboard resources.	Infrastructure failures require redundancy or onboard fallback capabilities; hybrid operation improves robustness.
System organization	Robot-centric architectures tailored to individual platforms.	Reference architecture supporting multiple infrastructure-based system realizations.

and transmit only compact scene representations, such as detected objects or occupancy maps, to the cloud. In this way, RAIL supports a continuum between fully onboard and fully infrastructure-based autonomy rather than prescribing a single implementation. The allocation of functionality between onboard and cloud execution is application dependent and should consider computational complexity, communication latency, bandwidth limitations, and safety requirements.

2.3. Comparison with conventional AMR architectures

Previous reviews provide broad and valuable overviews of AMR navigation, robotic systems for internal logistics, operational planning and control, and cloud robotics [1,3,4,29,30]. These studies are primarily organized around autonomy technologies, operational challenges, application domains, or cloud-computing capabilities. In contrast, the present paper adopts an architectural perspective on infrastructure-based AMR systems. Rather than organizing the discussion around individual technologies or functional components, RAIL proposes a reference architecture describing how sensing, perception, planning, and control may be distributed between robots, infrastructure, and cloud resources. This perspective enables systematic comparison of alternative system organizations and the trade-offs they entail. Furthermore, the architectural discussion is complemented by a long-term industrial deployment that illustrates one realization of the proposed architecture together with its practical benefits and limitations.

Table 1 summarizes the principal architectural trade-offs between conventional onboard AMR systems and the proposed RAIL reference architecture. Conventional AMR systems typically perform localization, perception, planning, and control using onboard sensors and computational resources [31,32]. In contrast, infrastructure-supported systems extend robot capabilities through infrastructure-mounted sensing and cloud computing, enabling collaborative perception and centralized fleet coordination. Rather than representing mutually exclusive approaches, the two paradigms occupy different points in the system design space. Conventional systems prioritize robot independence, whereas RAIL exploits shared sensing and computational resources to improve fleet-level performance while retaining onboard autonomy whenever required.

The primary advantage of infrastructure-based autonomy is that sensing and computational resources can be shared across the fleet. While installing infrastructure represents a significant initial investment, these resources may simultaneously support multiple robots, reducing the need to duplicate expensive sensors and high-performance computing hardware onboard every platform. Consequently, the marginal hardware cost of adding robots may decrease as the fleet grows, making infrastructure-based solutions particularly attractive for large industrial deployments.

Infrastructure support also enables capabilities that are difficult to achieve using purely onboard systems. Since infrastructure sensors can observe a substantially larger portion of the environment than any individual robot, the system can maintain a global representation of the factory, detect congestion before robots encounter it, and coordinate transportation tasks proactively. Furthermore, a globally referenced coordinate system can simplify deployment and reconfiguration. In our industrial implementation, for example, pickup and delivery locations can be modified by relocating fiducial markers, allowing production personnel to reconfigure transportation tasks without reprogramming the robots.

These benefits come at the expense of increased dependence on communication and infrastructure availability. Failures of wireless communication or infrastructure sensors may reduce perception quality or interrupt centralized planning. For this reason, we consider onboard autonomy and infrastructure support to be complementary rather than competing paradigms. Depending on the application, different functions may execute onboard or in the cloud, allowing the system to balance computational efficiency, robustness, and operational flexibility.

Beyond this comparison, it is also useful to position RAIL with respect to interoperability standards such as VDA 5050 [33]. VDA 5050 specifies a standardized communication interface between a fleet management system and mobile robots, enabling heterogeneous fleets from different manufacturers to operate under a common fleet controller. In contrast, RAIL describes how sensing, perception, planning, and control functionality may be distributed between robots, infrastructure, and cloud resources. Consequently, the two address complementary aspects of AMR system design rather than representing competing approaches, and a RAIL-based implementation may employ VDA 5050 as the communication interface between the fleet manager and the robots. While VDA 5050 standardizes the information exchanged through this interface, the realization of localization, perception, and navigation is outside the scope of the standard. RAIL complements this interface-level perspective by explicitly addressing how infrastructure-supported perception, shared-state representations, and cloud-based decision making can be integrated into the overall system architecture.

3. Key technologies

In this section, we review the key components of the reference architecture in Fig. 2. Specifically, the following subsections are dedicated to localization, environment perception, and fleet management and planning.

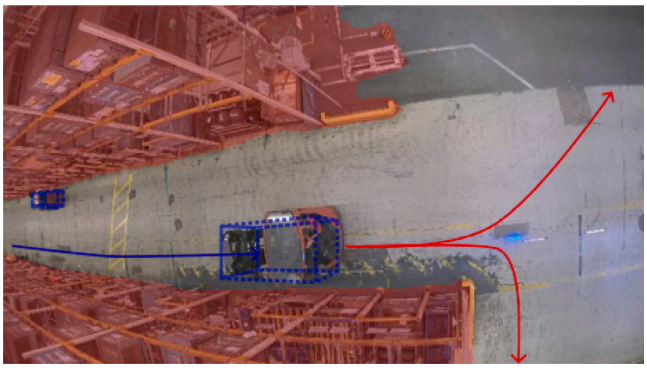


Fig. 3. Example perception results in a factory environment. Semantic segmentation is used to find static obstacles (red) and bounding box detection is used for vehicles (blue). Object tracking estimates the past trajectory of the vehicle (blue arrow), and motion prediction creates multiple hypothesis for the future movement (red arrows). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.1. Localization

For AMRs, localization involves estimating the robot's current position and orientation within its environment. Accurate localization is a fundamental prerequisite for enabling robots to navigate, perform tasks, and interact safely with their surroundings. While onboard sensors such as inertial measurement units (IMUs) and odometry can track the robot's motion in a local coordinate frame over short durations, additional sensing is required to determine the robot's global position. Unlike outdoor settings, where Global Navigation Satellite Systems (GNSS) are widely used, reliable GNSS signals are typically unavailable indoors. Consequently, various alternative indoor localization methods have been developed.

The most common approach for indoor localization is Simultaneous Localization and Mapping (SLAM) [34]. SLAM enables a robot to operate in unknown environments by using onboard sensors to simultaneously construct a map and localize itself within it. This problem has been extensively studied in the literature [35–37] and is supported by widely used open-source tools, such as the ROS 2 Navigation Stack [38,39]. SLAM techniques have been developed for both LiDAR sensors (e.g., GMapping [40], KartoSLAM [41], and Cartographer [42]) and camera-based systems, including monocular, stereo, and RGB-D setups [43]. Given the high computational demands of SLAM, cloud-based solutions have been proposed to offload processing from the robot itself [12]. Such approaches not only benefit individual robots but also support collaborative mapping in multi-robot systems [15–17]. Although recent advancements in SLAM has led to increased robustness and accuracy, SLAM-based localization remains challenging in highly dynamic environments.

In structured indoor environments, dedicated localization infrastructure is often installed to simplify the problem. Radio-frequency-based technologies such as RFID, Ultra-Wideband (UWB), Wi-Fi, Bluetooth, and Zigbee provide low-cost positioning, though typically with limited accuracy [34]. Combining such systems with SLAM or odometry/IMU data is a promising approach to enhance robustness and reliability [1,44]. Additional methods involve placing artificial landmarks, such as LEDs, reflectors, or fiducial markers, within the environment, which the robot can detect using its onboard sensors [3]. Localization is then achieved by estimating the bearing or distance to these targets. Another alternative is to use infrastructure cameras to monitor the robot from a fixed viewpoint. In such systems, a fiducial marker is often attached to the robot to facilitate accurate pose estimation [19,45].

3.2. Environment perception

Perception involves estimating the state the environment, such as the 3D positions and velocities of nearby vehicles and the road layout. Depending on the task, both parametric representations (e.g., bounding boxes and splines) or non-parametric representations (e.g., occupancy maps and voxel grids) are used. In the context of internal logistics, where both the robot and surrounding agents are restricted to planar motion, a bird's-eye view (BEV) representation is particularly well suited. Beyond estimating the current state of the environment, perception also involves reasoning about possible future states. This foresight enables the robot to make proactive decisions that enhance both safety and operational efficiency. Fig. 3 illustrates example output of the perception module consisting of static obstacles estimation alongside detection, tracking and motion prediction of dynamic objects. In the remainder of this section, we review commonly used perception methods described in the literature. While not universal, many of these approaches follow a modular pipeline that includes detection, tracking, and motion prediction.

3.2.1. Detection

Most literature on perception for AMRs and autonomous vehicles focus on onboard systems based on e.g., cameras, LiDAR and radar sensors [26,27,46]. Deep learning methods currently dominate most computer vision tasks, including object detection, depth estimation, and all types of scene segmentation (semantic, instance, panoptic), regardless of the specific sensor configuration. While progress is made on each of the mentioned tasks, creating a rich and dense representation of the environment remains challenging. Here, BEV perception has emerged as a promising solution, where e.g., multiple cameras [47], LiDAR [48] or a combination of both [49,50], are used to aggregate features on a virtual ground plane. Such a feature representation enables subsequent BEV segmentation and object detection, which can easily be consumed by downstream planning modules [51]. Despite the significant progress made, onboard perception remains vulnerable to occlusions and obstructions, leading to degraded performance in crowded environments.

Infrastructure-based perception literature mainly revolve around surveillance applications, where cameras are used to monitor environments such as traffic intersection or indoor facilities. Since the goal is to detect objects on the ground/floor, which is typically relatively flat, these applications benefit from simplified image-to-world coordinate mapping via precomputed calibration. Early methods used probabilistic frameworks to directly infer the ground plane positions based on detections in the images and precomputed transformation [52,53]. Recent works, however, learn dense BEV representations of the environment, which is particularly well suited for fusing information from multiple cameras viewing the same area [54].

With the advent of reliable, low-latency communication technologies, such as 5G, vehicle-to-vehicle (V2V) and vehicle-to-everything (V2X) perception has gained traction. These paradigms rely on information sharing between multiple autonomous agents and/or infrastructure devices, such as roadside cameras or LiDAR units, to overcome key limitations of onboard sensing, such as restricted range and blind spots [55]. In the robotics domain, multiple robots have been used for collaborative mapping of the environment [15–17]. Recent V2V applications in autonomous driving involve using multi-view images or LiDAR from connected vehicles to perform 3D object detections [56–58], BEV segmentation [57], or to predict the semantic occupancy status of 3D voxels [59]. In terms of V2X communication, [60] proposes a method for 3D object detection based on LiDAR observations from two connected vehicles and one roadside unit. While promising, these technologies are still in early development. Most current systems are limited to small-scale setups, and extensions to multiple vehicles and roadside units are rare. Moreover, creating training data for these systems is particularly challenging and expensive, as it typically requires detailed 3D annotations in complex sensor setups.

3.2.2. Multi-object tracking

Multi-object tracking (MOT) aims to estimate how object states (e.g., position, velocity and heading) evolve over time. This temporal information is essential for understanding the intent of dynamic agents and predicting their future motion, which is crucial for planning and decision-making [26]. Most modern approaches follow the tracking-by-detection paradigm [61–64]. Given detections from previous time steps, such methods aim to make associations across time to create tracks, estimate the state of each object based on the sequence of associated detections, and manage track initialization and termination as objects enter or leave the surveillance area.

One family of methods addresses these aspects under a probabilistic framework based on Random Finite Sets [65,66]. This approach provides a principled way to model uncertainties through motion and measurement models with recursive Bayesian updates. However, high computational complexity prohibits exact solutions, limiting performance. In practice, most methods instead rely on heuristics or learning-based methods to address parts of the tracking pipeline. For example, in scoring possible associations, measurement likelihood have been replaced by various affinity metrics, based on e.g., intersection-over-union (IoU) [61], distance [62,63,67], or appearance features [67,68]. Global optimal matching may then be computed with the Hungarian algorithm [61,67], but greedy methods are also common [62,63]. Moreover, initialization and termination of tracks are often based on simple count-based rules [61–63].

Tracking-by-detection has been extensively studied in camera-based surveillance [67,69–71], and in LiDAR or camera-based autonomous driving [61–63]. Recent methods go beyond pure detection-based tracking: [72,73] perform joint detection and tracking, while [74] introduces a transformer-based tracking-by-attention paradigm. Recent methods also study V2X-enabled tracking. [75] propose sharing detections between vehicles to enhance subsequent tracking-by-detection robustness. [76] instead fuse features from an infrastructure and onboard LiDAR and perform transformer-based joint detection and tracking. Designing high-performing multi-object tracking methods that are robust and computationally tractable remains an open research challenge, especially in the emerging field of V2X perception.

3.2.3. Motion prediction

Motion predictions aims to estimate the future location of relevant dynamic objects based on the detection and tracking results. Historical motion profiles of the target and nearby agents, along with environmental layout, are commonly used. Traditional methods use established motion models of dynamic obstacles for prediction, such as the constant velocity model [77], the social force model [78], and the velocity obstacle model [79]. However, traditional methods exhibit notable limitations. They struggle to account for complex interactions and maneuvers of target objects without a predefined rule base, and face difficulties in generating accurate predictions in complex situations.

Advancements in deep learning have enabled neural networks to model environmental context and agent interactions while accounting for uncertainty, making them well-suited for dynamic environments. Social pooling [80] is a technique that captures interactions among agents by aggregating their features to infer collective behavior, allowing the network to learn motion patterns while explicitly considering these interactions. CNNs are widely used to extract spatial information from the environment [81–84], thereby enhancing the network's ability to perceive the layout of the workspace. This is crucial to make adaptive predictions when the surroundings are dynamic and the layout may change over time. Recently, the transformer attention mechanism [85,86] and inverse reinforcement learning [87] have been used to capture the intricate dependencies between agents and their environments. Still, generating predictions that are accurate and adequately describe the (multi-modal) uncertainty about the future motion of objects remain difficult in complex environments.

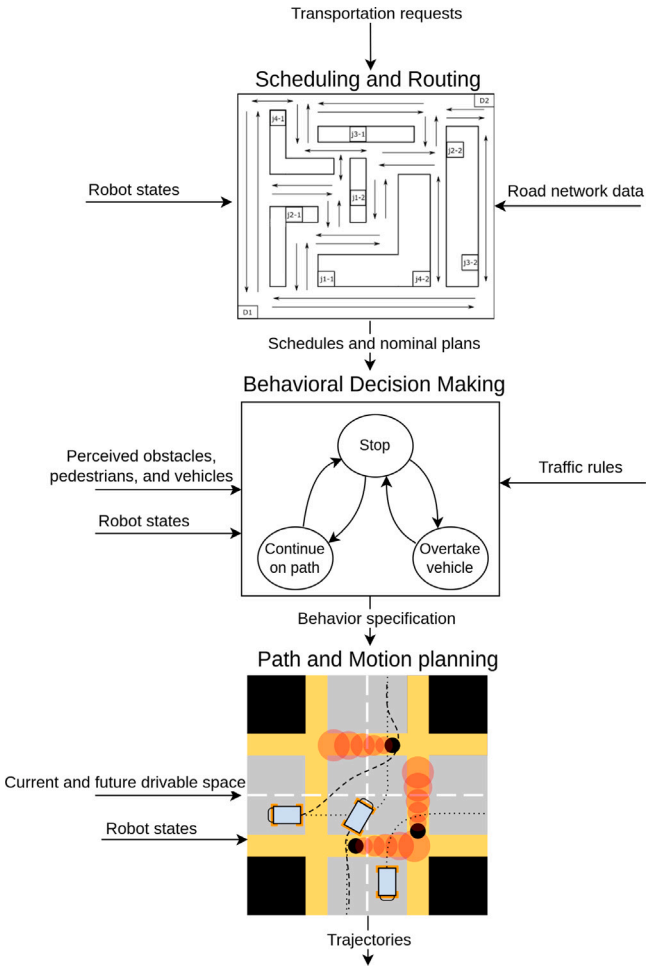


Fig. 4. Overview of the fleet management and planning framework. Arrows from the left correspond to output from the perception module, while arrows from the right are user specifications. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.3. Fleet management and planning

In the autonomous driving community, deciding the next action for the vehicle is typically accomplished with hierarchical pipeline consisting of global route planning, behavioral decision making, local motion planing, and control [25,27,88,89]. Similar ideas can naturally be applied to the control of AMRs, although, we are considering an entire fleet rather than a single vehicle. The rationale behind the hierarchical decomposition of the problem is two-fold. First, the uncertainty about the environment grows rapidly with time, which makes it feasible to explicitly consider possible futures only over short time horizons. Second, details that are central to short-term planning, e.g., exact vehicle dynamics and predicted motion of nearby vehicles, may be disregarded in long-term planning as they are unlikely to effect the results.

Fig. 4 illustrates a hierarchical planning pipeline for AMRs. At the highest level, scheduling and route planning assign tasks to the robots based on the road network and current transportation demands. The considered time horizon for this problem is typically hours. Therefore, only nominal plans can be provided by this layer, which is then supplemented with real-time decision making at lower levels. Behavioral decision making is responsible for determining the course of action over a shorter time horizon, typically not longer than a few minutes. The sequence of action is based on current observations of the environment,

e.g., overtake the preceding vehicle, continue on route to the depot station, and come to a stop. The selected behavior is often manifested in a reference path provided by a path planner. As available paths are continuously changing in a dynamic environment, path planning is a central component in the pipeline. Subsequently, the motion planning layer computes a trajectory that progresses the robot along the reference path in order to achieve the selected behavior. The trajectory typically spans only a few seconds, which allows for explicit reasoning about possible future states of nearby dynamic objects to produce feasible trajectories. Finally, a local feedback controller is employed to follow such trajectory.

In the following, we review relevant literature on scheduling and route planning, behavioral decision making, path planning, and motion planning. The low-level control is excluded since it is not specific for our application and has already been covered by a large body of literature. It is noteworthy that the distinction between the different layers is somewhat vague, and methods that treat, for example, behavioral decision making and motion planning jointly are common.

3.3.1. Scheduling and route planning

Scheduling and routing is typically formulated as a Vehicle Routing Problem (VRP), a class of NP-hard combinatorial optimization problems. Optimization is performed over a graph derived from the predefined road network. The task is to determine which edges each robot should traverse, and at which time, to fulfill transportation jobs. The edge weights typically correspond to distance or estimated travel time. The graph is often directed to account for one-way and two-way roads, and may include features such as edge capacity to manage congestion and depot stations that represent robot start, end, or charging locations. Time windows can also be incorporated to ensure that robots are at specific locations at designated times, which is a common requirement in industrial applications [90]. Commonly, the optimization objective is a function of cumulative traveling distance, tardiness, or level of congestion [91].

Due to the high computational complexity, solving the scheduling and routing problem simultaneously in a single, monolithic optimization formulation is only practical for relatively small instances. Feasible problems sizes are limited to a few robots and tasks, and a graph with a few dozen nodes. In such cases, both approximate methods [92] and exact algorithms [93] can be used effectively. For larger systems, scheduling and routing may be treated separately to reduce the computational complexity. Specifically, iterating between first solving the routing problem and then the scheduling problem is a promising approach [94]. Alternatively, routes and schedules can be computed sequentially for one AMR at the time [95]. While these approaches allow for faster computation and dealing with larger problem instances, they cannot guarantee global optimal solutions. Another approach to compute routes and schedules is to use machine learning (ML) algorithms [96]; the most investigated method, as well as the most promising, is to train reinforcement learning agents end-to-end [97,98] to either drive a single AMR, or the entire fleet. On the other hand, ML has also been used in combination with analytical methods [99]; in the modeling phase, ML can be used to estimate model parameters related to features of the problem from historical data, while in the solving phase, ML can be used to quickly generate initial solutions that can be further improved by non-learning methods.

3.3.2. Behavioral decision making

To account for dynamic elements in the environment, such as nearby traffic and pedestrians, an online planner is required. Typically, planning is based on a domain model that includes possible environmental configurations (states), action definitions with preconditions and effects, and the goal specification [100]. By searching the state and action space of the model, the planner finds the sequence of actions needed to reach the next scheduled goal.

In classical planning, the Planning Domain Definition Language PDDL offers a standard for modeling. PDDL defines types, predicates, and actions [101], and is widely used for structured problem representations. It also has extensions that support temporal aspects of planning [102]. For reactive systems, wherein asynchronous events ought to trigger certain responses, event-driven formalisms such as Event-Condition-Action (ECA rules) [103] or Extended Finite Automata (EFAs) [104] are often more suitable.

Planning techniques vary depending on the complexity of the problem. For smaller problems, explicit state-space search methods such as Breadth-First Search are common [105]. For larger state spaces, symbolic methods based on Binary Decision Diagrams (BDDs) [106] or SAT-based planning, which leverages recent advancements in SAT solvers by casting the problem as Boolean Satisfiability [107], are more effective. To further manage complexity, specifications like Linear Temporal Logic (LTL) can be used to define desired properties over execution paths [108]. This helps prune the search space by eliminating solutions that violate critical constraints [109].

A key remaining challenge is creating the domain model. Manual definition is time-consuming and error-prone, especially in complex or dynamic environments [110]. Recent work attempts to learn such models through interaction with a simulator or the real-world system using techniques such as active learning [111]. However, reliably learning accurate and comprehensive models remains an open research problem.

3.3.3. Path planning

The path planning problem concerns finding either the shortest or a collision-free path through an environment. A path $\mathcal{P}$ can be defined as an ordered set of connected points or a spline in space, containing an initial point $\mathcal{P}_{\text{start}}$ and a final point $\mathcal{P}_{\text{end}}$. Path planning approaches are commonly categorized as graph-based, sampling-based, or numerical optimization methods [112].

Graph-based methods like Dijkstra's and A* require discretization of the environment into a graph-like structure, and is widely used in static environments [112]. However, when the environment changes, algorithms such as A* must typically recompute the entire path, which limits their efficiency in dynamic settings [113]. D* and its derivatives [114] address this limitation by updating only the affected portions of the path when changes are detected, significantly reducing the computational burden in dynamic or partially known environments.

Sampling-based methods such as Probabilistic Roadmap and Rapidly-Exploring Random Trees [115] operate directly in continuous configuration spaces by randomly sampling states and incrementally building connectivity graphs or trees. These methods are particularly effective in high-dimensional planning problems due to their ability to explore large state spaces efficiently [116], and offer probabilistic completeness despite generally lacking guarantees of optimality.

Numerical optimization methods such as Ant Colony Optimization (ACO) and Genetic Algorithms (GA) offer flexible, heuristic-driven approaches to path planning. These methods typically require discretization of the search space, for example by modeling the environment as a grid or waypoint graph. ACO uses pheromone-based exploration over such discrete representations to discover globally efficient paths, with recent variants improving convergence speed and robustness in dynamic environments. GA evolves path solutions via fitness-based selection, often optimizing both path length and smoothness within discretized configurations [115]. Although careful parameter tuning is required and these methods are less suited for strict real-time use, they are effective in offline planning frameworks.

3.3.4. Motion planning

Collision-free motion planning, also referred to as trajectory planning or local path planning, computes a dynamically feasible trajectory that safely guides a robot from its current state to a desired goal. The trajectory must respect the robot's kinematic and dynamic constraints, avoid both static and dynamic obstacles, optimize cost terms such as

energy consumption, safety margins, execution time, and trajectory smoothness, and adhere to a reference path provided by a global planner [117]. As shown in the bottom part of Fig. 4, three AMRs navigate a shared intersection alongside pedestrians, following reference paths (dotted lines) while generating locally feasible trajectories (dashed lines) that react to predicted obstacle motion. Effective planning in such dynamic environments depends both on the chosen algorithm and how obstacles are represented. The translucent red ellipses in the figure depict a probabilistic obstacle model that captures both position and uncertainty over time. Obstacle models may be parametric (e.g., geometric primitives like ellipses and bounding boxes) or non-parametric (e.g., occupancy grids, Euclidean Signed Distance Fields, or Truncated Signed Distance Fields), and either deterministic or probabilistic depending on how uncertainty is treated [113,117,118]. Motion planning methods are commonly categorized into four classes: reactive, optimization-based, learning-based, and hybrid methods, based on how they process information and compute actions.

Reactive motion planning methods compute control commands or trajectories in response to current state and local sensor inputs, without relying on long-horizon predictions. These approaches are typically fast and well-suited for dynamic environments. Notable examples include the Dynamic Window Approach (DWA), which searches a window of admissible velocities to select the one that maximizes an objective function while ensuring safe stopping distances, and the Timed Elastic Band, which optimizes a time-parameterized trajectory using soft constraints for obstacle avoidance [117,119].

Optimization-based approaches formulate motion planning as a constrained optimization problem, where the objective is to compute a feasible and cost-efficient trajectory. A prominent example is Model Predictive Control (MPC), which uses a predictive model of the robot's dynamics to iteratively optimize control inputs at each time step over a finite time horizon. MPC incorporates motion constraints such as dynamics, input bounds, and safety requirements, and evaluates a cost function that typically balances tracking performance, smoothness, and control effort. The receding horizon structure makes MPC suited for dynamic and uncertain environments [118]. However, solving nonlinear optimization problems in real time imposes a significant computational burden. To alleviate this, many implementations leverage numerical approximations, such as Sequential Quadratic Programming (SQP), or warm-start techniques using precomputed feasible trajectories [118].

Learning-based motion planning includes both reinforcement learning (RL) and imitation learning (IL). RL learns a policy through trial-and-error interactions with the environment, often in simulation, optimizing cumulative reward signals that encode motion objectives and constraints. Imitation learning, on the other hand, derives policies by mimicking expert demonstrations. These methods are data-intensive and typically require extensive training, but offer good adaptability to dynamic and uncertain environments [120].

Hybrid approaches combine different planning paradigms to offset the limitations of individual methods. For example, integrating the global optimality of MPC with the real-time reactivity of RL improves robustness in uncertain environments [121,122]. Similarly, combining DWA with RL enables reactive safety while preserving learned behavior [123].

Despite recent progress, motion planning for AMRs remains challenging, particularly when accounting for the robot's kinematics and dynamics alongside timing and collision constraints. Early efforts in this area include trajectory planning based on Signal Temporal Logic (STL) specifications, combined with control strategies such as model predictive control [124,125] or control barrier functions [126].

Ensuring real-time feasibility under dynamic and nonlinear constraints remains difficult, especially in unstructured environments [127]. Uncertainty from imperfect sensing and prediction complicates safety assurance, with deterministic methods often being overly conservative and probabilistic ones lacking formal guarantees [127]. Scalability is another issue, as high-dimensional or multi-agent scenarios strain most planning algorithms [113]. While learning-based methods promise adaptability, they require extensive data, struggle to generalize, and remain hard to interpret or verify [120]. Bridging classical and learning-based approaches while ensuring robustness and safety is a key direction for future research.

4. Industrial evaluation

We have implemented and deployed an infrastructure-based AMR system in an industrial setting at Volvo Group Truck Operations (AB Volvo). The experiences gained from the design, deployment, and long-term operation of this system motivated the development of the proposed RAIL architecture. Consequently, the implementation should be regarded as one realization that inspired RAIL rather than as a complete implementation of the reference architecture. Whereas RAIL is intended to accommodate a broad spectrum of infrastructure-based AMR systems with varying degrees of onboard autonomy, the deployed system represents one particular design point in this space.

In the deployment, six robots are used for transporting mufflers from a pre-assembly area to the assembly line. The cycle time is approximately 7 min, resulting in up to 130 transport operations per day. Each transport task involves a 150 m drive through a mixed-traffic environment shared with manually operated vehicles and pedestrians, necessitating robot behaviors capable of handling dynamic and uncertain conditions.

The primary design objective of the deployed system was to minimize the complexity and cost of the robots by reducing the need for expensive onboard sensors and computational resources. Consequently, localization, perception, and planning were offloaded to a centralized infrastructure consisting of an on-premise compute cluster and ceiling-mounted cameras. These cameras jointly provide a top-down view of the entire operating area and are used for both robot localization and environmental perception. Unlike the general RAIL architecture, which supports different allocations of functionality between onboard and cloud execution, this implementation relies almost entirely on infrastructure-based perception and planning, with the robots performing trajectory tracking, low-level control, odometry fusion, and emergency braking onboard.

Fig. 7 illustrates the system architecture. The infrastructure cameras send image streams over Ethernet to the local compute cluster. These images are processed to localize the robots and to build an occupancy map of the environment. Based on the current robot positions, the occupancy map, transportation requests, and a pre-defined road network, a fleet manager assigns tasks and nominal paths to the robots. A path planner then generates motion plans, which are transmitted via Wi-Fi together with a shared state to the robots.

In this implementation, the shared state represents a limited realization of the abstraction introduced in Section 2.2. Specifically, it contains the robot's current global pose and the current motion plan. The robot pose is updated by the infrastructure using observations from the ceiling-mounted cameras at approximately 1 Hz, while each robot continuously refines its own pose estimate using onboard odometry at approximately 10 Hz. The shared state does not include a complete world model or the states of all dynamic objects. Instead, the occupancy map used for planning is maintained centrally within the infrastructure, while the robots receive only the information required to execute their assigned tasks.

The robots execute the received motion plans using low-level controllers, relying on motor odometry as feedback. For safety, an onboard short-range RADAR and a bumper stop activate emergency braking.

The following subsections describe the hardware and software architecture of the system in more detail.



Fig. 5. The transportation robot. The scene was reconstructed using Gaussian splatting [129].

4.1. Hardware and middle-ware

4.1.1. Robots

Fig. 5 shows the robot platform, including:

- A safety radar and a bumper stop for collision protection,
- A 48 V battery,
- Two motors controlled via a CAN interface,
- Two ESP32-C3 microcontrollers for control and communication,
- A display, lamps, and control buttons,
- A unique ArUco marker [128] (4 × 4 format, 100 mm size) for localization via the ceiling cameras.

Notably, the robots are not equipped with LiDAR or cameras, which are commonly used for onboard perception and SLAM. Instead, localization and obstacle detection are handled entirely by the ceiling-mounted cameras. Robots receive navigation instructions via Wi-Fi, in the form of simple motion commands and waypoints. The robot firmware is written entirely in Rust, using the async embassy framework. The robots can also be operated manually. A button on the handle switches the robot to manual mode, allowing human operators to move it as needed. When switched back to autonomous mode, the robot automatically returns to its assigned path.

4.1.2. Infrastructure cameras

To enable localization and perception, Power-over-Ethernet (PoE) IP cameras were installed in the factory ceiling. In total, fifteen cameras where installed, which together cover the entire robot operating area. Their layout is shown in Fig. 6. Most cameras were mounted at a height of approximately 8 m and oriented in a top-down configuration, each covering about 60 square meters of floor area. The cameras were calibrated using the OpenCV toolkit, based on the pinhole camera model and the method described in [130], with a custom-made 2 × 1 meter calibration target. During operation, each camera provides two image streams to the compute cluster:

- Low-resolution (640 × 360 px) images at 10 Hz, used for obstacle detection;
- High-resolution (3840 × 2160 px) images at 1 Hz, used for robot localization.

The cameras are not hardware-synchronized, which means that the images from different cameras are captured with slight time difference.

4.1.3. Compute cluster

A kubernetes cluster was used installed on simple bare metal nodes including one Nvidia A4000 GPU each. The semantic segmentation inference can handles up to 25 cameras per GPU at 10 fps per camera. Most of the code is implemented using rust, making it resource efficient and stable. The cluster is running in an air-tight factory environment in the normal factory network.

The code structure is simple, using a redis data base as a shared state and the robots communicates with the cluster via the factory wifi network using a simple udp-protocol.

In this deployment, computational tasks are statically allocated to the on-premise compute cluster rather than dynamically migrated between robots and infrastructure. Localization, perception, fleet management, and path planning execute entirely on the infrastructure, while the robots perform only trajectory tracking, low-level control, odometry fusion, and emergency braking. This static allocation simplifies system integration and provides deterministic computational performance for the deployed application, although more adaptive task allocation strategies represent an interesting direction for future work.

4.2. Software

4.2.1. Localization

Localization is based on detecting the unique ArUco marker attached to each robot. The high-resolution images from the infrastructure cameras are processed using the OpenCV ArUco library [128,132] to detect these markers. Given that the markers lie on a known horizontal plane (parallel to the factory floor), and at a fixed height, the 2D image coordinates of the marker corners can be mapped to 3D coordinates using the established camera calibration. From these corner positions, the robot's Cartesian coordinates (x, y) and heading θ are computed directly.

The global localization accuracy is approximately 10 cm, where the dominant source of error arises from calibration inaccuracies between cameras in the factory-wide coordinate system. As expected, these errors increase with the distance between calibration landmarks. However, the relative calibration between neighboring cameras is considerably more accurate. Consequently, local positioning tasks, such as docking at pickup or delivery stations observed within the same camera, achieve an accuracy of approximately 10 mm, which has proven sufficient for reliable loading and unloading operations.

Processing the high-resolution camera images required for ArUco detection is computationally demanding, resulting in an update frequency of approximately 1 Hz and an end-to-end latency of up to 600 ms. To compensate for this delay, each robot continuously fuses the infrastructure-based pose estimates with onboard wheel odometry. The onboard odometry provides smooth real-time pose estimates between successive global updates, while the infrastructure measurements periodically correct accumulated drift, which is essential for maintaining accurate localization during long-distance navigation.

4.2.2. Perception

The factory floor is discretized into a uniform grid, with each cell representing a 5 × 5 cm area. The perception system classifies each cell as free or occupied based on semantic segmentation. Specifically, low-resolution images from each camera are processed using a binary semantic segmentation model to distinguish obstacles from free space. These 2D pixel maps are transformed to the ground plane using a precomputed homography based on the established camera calibration. Since this homography merely maps the image plane to the ground plane, it disregards the true depth of each pixel in the image. However, since the cameras are mounted at a considerable height, the varying height of the objects in the scene only affects the results modestly.

Once the per-camera occupancy maps have been computed, they are fused into a global map. Where cameras have overlapping views, the global map is based on only one of the cameras (typically the closest one). Note that since the cameras are not hardware-synchronized, some errors arise due to time shifts between the different cameras. However, the impact is small given the high processing rate (10 Hz).

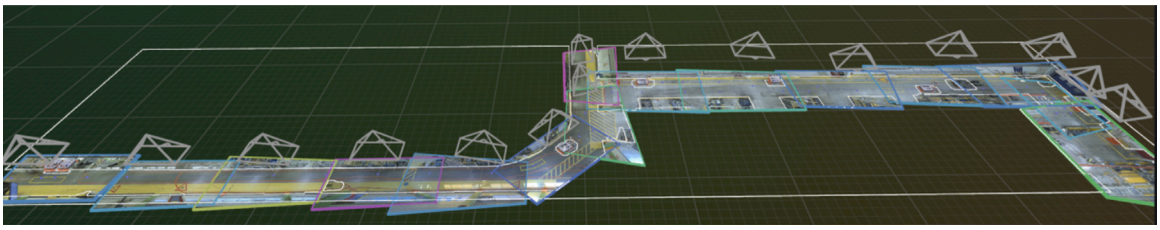


Fig. 6. Rerun [131] visualization of the deployment area and camera positions. Each of the 15 cameras cover an area of roughly 12 × 6 m.

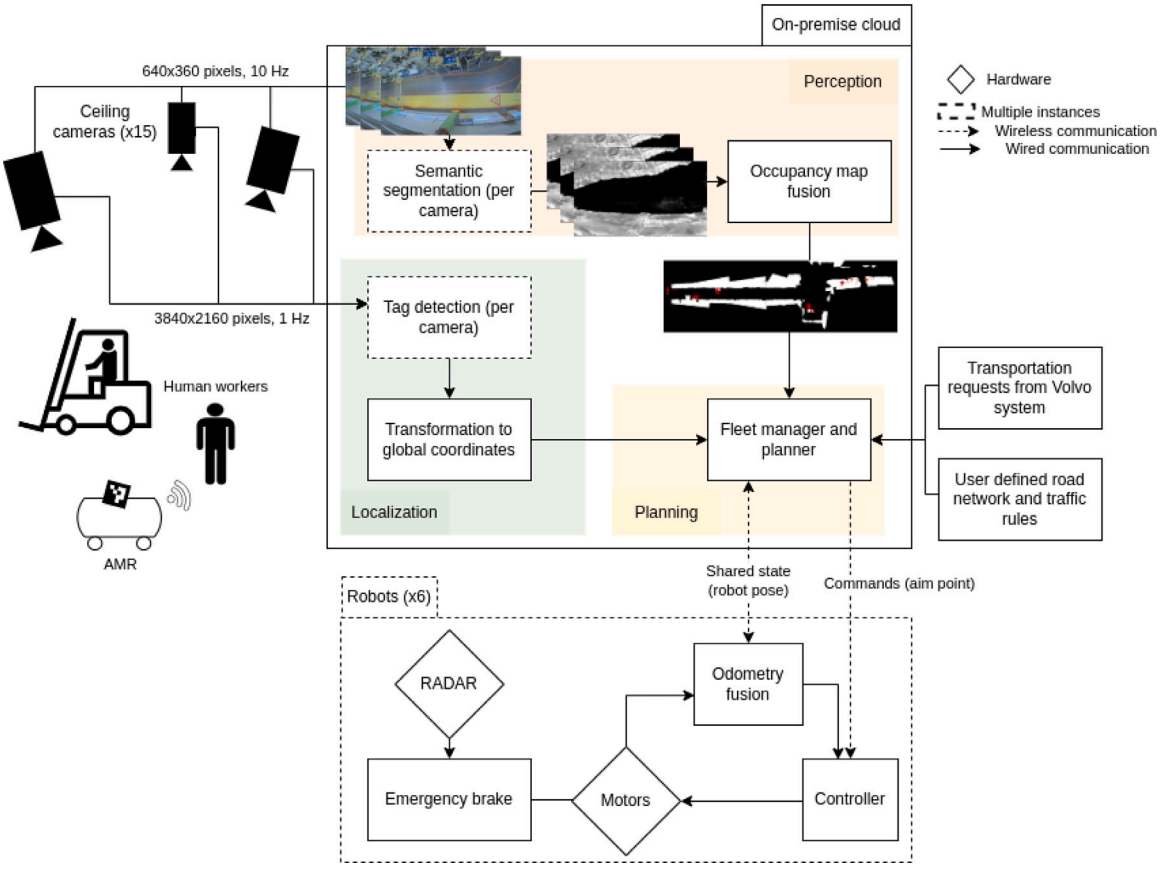


Fig. 7. System architecture of the industrial implementation at Volvo.

The end-to-end latency from image acquisition to transmission of updated motion commands to the robots is approximately 150 ms, with the dominant contribution arising from image acquisition within the cameras. At the maximum robot speed of 1.5 m/s, a robot travels approximately 22.5 cm during this interval before responding to newly detected obstacles. This response time is comparable to that of human operators and has proven sufficient for safe interaction in the mixed-traffic industrial environment. Furthermore, the infrastructure-based perception is complemented by onboard emergency braking based on a short-range RADAR and a bumper stop, providing an independent safety layer should infrastructure perception or communication be delayed or unavailable.

The semantic segmentation model has not been evaluated independently against manually annotated ground truth, as acquiring representative pixel-level annotations in a large-scale industrial environment is prohibitively labor-intensive. Instead, the perception system has been validated at the system level through long-term deployment, where it has demonstrated sufficient performance to support reliable obstacle detection and safe robot navigation under normal operating conditions.

4.2.3. Fleet management and planning

The deployed system supports a single transportation task type: transporting mufflers from a pickup station to the assembly line and returning. Scheduling is handled using a simple queue-based mechanism. At the start of each day, the three robots queue at the depot. When a new transport request arrives, the next available robot is dispatched. Execution of each task follows a predefined finite-state machine (FSM), with states and transitions manually defined based on application-specific conditions (e.g., is the robot kitted? is the path clear? are there other vehicles in the intersection?). These checks rely on the perception system and logical evaluations. Navigation is based on the A-star algorithm, which computes paths over a manually defined road network. Robots follow these paths using a simple motion controller that "aims" for a point several meters ahead, adjusting speed via ramp profiles based on distance. Heading is regulated via a feedback controller using heading error.

In summary, the robot behavior is intentionally kept simple. Robots stop or slow down when obstacles block their path and yield to higher-priority vehicles. They do not re-plan around obstacles or leave their assigned route. As a result, factory workers must keep robot paths clear for smooth operation. However, the robot may be manually moved

off its path, in which case it will automatically navigate back to the assigned route when reactivated.

4.3. User experience evaluation

While the technical performance of localization, perception, and planning is fundamental to infrastructure-based AMR systems, successful industrial deployment also depends on how human operators perceive and interact with the system. Since RAIL is intended for collaborative environments in which robots operate alongside human workers, evaluating the user experience (UX) provides complementary insight into practical deployment challenges that are not captured by technical performance metrics alone. The following evaluation therefore considers our industrial implementation of the proposed architecture from a human-centered perspective. Its purpose is not to evaluate the RAIL architecture in isolation, but to identify deployment challenges and design considerations for infrastructure-supported AMR systems operating in shared industrial environments.

As mobile and collaborative robots are integrated into industrial settings, there is a growing need for human-centered design and interaction [133]. This evolution underscores the significance of user experience (UX) in human–robot collaboration, focusing on trust, comfort, and communication alongside traditional concerns for safety and efficiency [134]. Key UX challenges faced by workers in shared workspaces with mobile robots were investigated in [135] to inform human-centered design strategies that enhance safety, productivity, and well-being. A post-evaluation was conducted two months after the AMR field test, using the UX Questionnaire [136] and open-ended interviews with eight participants (four operators and four truck drivers) who worked closely with the robots. The UX evaluation covered six aspects: effectiveness, efficiency, safety, sustainability, pleasure, and predictability [135].

Effectiveness was partly satisfactory. Operators found the robot helpful for material delivery and reducing manual tasks, while truck drivers felt it added complexity and delays. Efficiency was rated as partly satisfying; operators appreciated its ease of use and time savings, but truck drivers struggled with unclear behavior, detection issues, and lack of training. Both groups needed better handling of unexpected situations and more support during onboarding. Safety was unsatisfactory, especially for truck drivers facing visibility issues, collision risks, and unclear statuses. Operators generally felt safe but were concerned about the robot's ability to detect obstacles, such as forklifts. Sustainability ratings were low due to cognitive strain on truck drivers and worries about long-term conditions. Operators found it slightly acceptable but noted ergonomic deficiencies and the need for constant monitoring. Despite some frustrations, pleasure received the highest rating, as both groups found the robots interesting and innovative. Predictability feedback was mixed; operators found the robots largely predictable, while truck drivers encountered unpredictable behavior in non-standard scenarios.

Key issues included challenges in both the physical and cognitive work environments. Physical problems encompassed difficulties in detecting the robot and collision risks. Challenges in the cognitive work environment, such as unclear status, lack of training, increased task demands, workflow disruptions, authority ambiguities, and unclear benefits, further obstructed acceptance. Concerns were also raised regarding the scalability of the system and potential negative impacts on truck drivers' future roles. Overall, while the robot offers innovation and some operational support, improvements are needed to fully assist truck drivers and operators [135].

Overall, the evaluation revealed substantially different experiences between assembly operators and truck drivers, illustrating that the same robotic system may affect different user groups in fundamentally different ways. These observations highlight the importance of incorporating human-centered design principles throughout the development of infrastructure-based AMR systems. In particular, the study indicates that technical improvements in perception and planning alone are insufficient for successful deployment. Future systems should also provide mechanisms for communicating robot intentions, improving robot visibility and predictability, supporting user-specific training, and establishing clear traffic management strategies that facilitate safe and efficient collaboration between humans and autonomous robots. Although these findings originate from one realization of the proposed RAIL architecture, they identify important system-level design considerations that are broadly applicable to infrastructure-supported AMR systems deployed in shared industrial environments.

5. Challenges and future outlook

To meet the industry's growing demands for flexibility, autonomous mobile robots (AMRs) must operate safely and efficiently in dynamic and uncertain environments, often alongside human workers. Infrastructure-based sensing and computing provide a promising path forward by extending perception, increasing computational capacity, and enabling global optimization across fleets. However, fully realizing the benefits of such systems requires advances across a wide range of technical domains. In what follows, we group the key challenges into four thematic areas: core autonomy and intelligence, infrastructure challenges, system-level considerations, and human-centered design. For each area, we outline open problems and future research directions.

5.1. Core autonomy and intelligence

5.1.1. Localization

Localization is widely recognized as a key challenge for AMRs, particularly in dynamic and unstructured environments. Infrastructure-based methods, such as those relying on ceiling-mounted cameras, can offer high localization accuracy and are relatively straightforward to implement in controlled settings. However, issues such as occlusions, sensor faults, or limited coverage can disrupt such localization methods, making fallback strategies like onboard SLAM essential. Meanwhile, onboard SLAM faces its own challenges. Performance typically degrades in environments that are highly dynamic, cluttered, or subject to frequent layout changes. As a result, infrastructure-sparse settings remain difficult for current systems, and advancing localization in such contexts is an important direction for future research.

A promising direction is to combine infrastructure-based and onboard localization. This includes collaborative and cloud-based SLAM methods and the integration of dedicated localization technologies such as Ultra-Wideband (UWB), RFID, Wi-Fi, Bluetooth, Zigbee, artificial landmarks, and infrastructure-mounted cameras. However, fusing these modalities introduces new technical challenges in data fusion, temporal and spatial synchronization, and real-time communication. Addressing these will be essential for reliable, scalable localization in industrial AMR deployments.

5.1.2. Perception

Perception in autonomous systems largely depends on deep learning models for tasks such as feature extraction and object detection. These models are typically trained in a supervised manner, making their performance highly dependent on the availability of large labeled datasets, which are time-consuming and costly to create. A key challenge, therefore, lies in developing generalizable perception models that can perform well with limited labeled data. Approaches such as semi-supervised learning and unsupervised domain adaptation aim to address this by leveraging small labeled datasets alongside larger collections of unlabeled data. More recently, foundation vision models have shown promise in mitigating these limitations by providing universal feature extractors that can simplify various downstream tasks.

Additional challenges arise when considering the specific application, involving on-premise cloud processing of heterogeneous data streams from both infrastructure and onboard sensors. Such systems

must handle spatial and temporal misalignment caused by calibration errors and communication delays. Moreover, due to limited wireless bandwidth, it is often impractical to transmit raw sensor data to the cloud. Instead, perception algorithms must be designed to operate under communication constraints and prioritize information that contributes most effectively to the system's performance.

Another critical challenge is the quantification of uncertainty, which is essential for robust planning and decision-making. Aleatoric uncertainty arises from inherent factors such as sensor noise and occlusions in the environment. Epistemic uncertainty, on the other hand, stems from model limitations, such as insufficient training data or poor generalization to novel scenarios. Predicting the future state of dynamic environments introduces additional uncertainty, particularly when human behavior must be anticipated. These uncertainties are especially difficult to model when fusing data from multiple modalities and perspectives, as their interactions can affect the overall confidence in non-trivial ways.

More broadly, ensuring robust perception in dynamic environments with frequent occlusions and unpredictable events remains a major challenge. The perception pipeline typically includes tightly coupled components such as detection, tracking, and motion prediction. Errors in early stages can propagate and compromise overall system performance. Improving robustness across this interdependent pipeline is an active and ongoing area of research.

5.1.3. Fleet management and planning

Challenges in fleet management and planning span the entire decision-making stack of AMRs. At the high level, scheduling and route planning must cope with growing computational complexity as fleet sizes and routing alternatives increase. These plans also need to remain adaptable to unforeseen disturbances. Ongoing research focuses on developing scalable algorithms capable of efficient rescheduling and robust plan repair in response to disruptions. At the mid level, behavioral decision making depends on accurate and comprehensive models of the environment and robot behavior. Specifying these models manually is time-consuming and error-prone, while learning them from data presents its own set of challenges. At the low level, path and motion planning in dynamic, unpredictable environments remains difficult. Challenges include computational efficiency, safety guarantees, and handling uncertain perception data.

Beyond these layer-specific issues, the hierarchical structure itself introduces fundamental limitations. Decisions at each level are often made in isolation, based on abstracted and locally available information, without full awareness of constraints or uncertainties encountered at lower levels. While this architecture supports modularity and computational tractability, it can lead to suboptimal or fragile behavior, especially in uncertain environments where robustness is critical. The abstraction process tends to omit essential context, such as real-time disturbances, resource contention, or spatio-temporal variability. This hinders the higher layers' ability to anticipate whether their plans will remain feasible or resilient downstream. As each level optimizes its own objectives independently, the compounded effects can yield plans that are neither robust nor valid for the system as a whole. A key challenge lies in dynamically updating these abstracted representations at runtime. For instance, if a route consistently takes longer than expected, the high-level planner must integrate that feedback to avoid repeated failures in future plans.

5.1.4. Lifecycle Management of ML components

Developing and deploying dependable machine learning (ML) faces significant challenges related to the inherent uncertainty of ML models. These include collecting and preparing representative datasets, mitigating model epistemic uncertainty (resulted from functional insufficiency) and managing aleatoric (irreducible) uncertainty introduced by factors such as annotation errors, sensor noise, and environmental condition variations. Furthermore, unlike traditional software where full test coverage can be achieved via code scanning, scenario-based testing seems to be the only viable approach for ML based systems. However, it requires large and diverse datasets that represent the set of test scenarios, requiring significant data collection and annotation efforts. Recreating hazardous scenarios for data collection can be challenging or sometimes infeasible. Even with enormous data collection investment, achieving sufficient coverage of all possible data variations remains difficult.

A promising approach is to augment real-world datasets with synthetic data generated by simulation, increasing variations and coverage. A key challenge is to generate diverse and realistic scenarios along with sensor data, such as images and LiDAR scans, which can allow for closed-loop testing of the system. Recent techniques based on Gaussian Splatting [137] and NeRFs [138] hold significant potential as they can be used both to reconstruct real-world scenarios and generate realistic sensor data from novel view-points. Despite the potential of facilitating data collection and reducing annotation costs drastically, the sim-to-real gap remains a challenge. More research is required to bridge the simulated and real-world domains for perception and decision making tasks alike. This involves improvements in multiple aspects: making the simulation more realistic, and improving model generalization capabilities through model design and training methods.

Finally, regardless of how well data collection and model engineering have been performed, unknown/unsafe scenarios will inevitably arise during system operation. Therefore, ML based systems must be equipped with supervisory monitoring mechanisms to timely detect unforeseen operational hazardous risks and support continual data collection for future model updates. Unfortunately, this introduces a new challenge: training on new data risks compromising previously certified performance, requiring rigorous and costly verification and validation (V&V) activities for each model release.

5.2. Infrastructure challenges

5.2.1. Communication and networking

Seamless wireless communication between robots, infrastructure sensors, and edge computing clusters is essential for the safe and efficient operation of infrastructure-based AMRs. Since the system transmits time-critical data, it is highly sensitive to network disruptions and delays. In particular, unpredictable variations in latency, known as jitter, pose significant challenges for perception and planning systems and must be carefully mitigated. Additionally, because AMRs typically move across large factory areas, maintaining consistent wireless coverage and seamless handover between access points is critical. This is especially difficult in dense industrial environments filled with machinery, assembly lines, and vehicles, all of which can cause signal interference.

Technologies such as 5G and Wi-Fi 7 are designed to address these challenges. 5G enables ultra-reliable and low-latency communication (URLLC) through features such as millimeter wave (mmWave) and flexible numerology, making it suitable for real-time robotics applications. Its support for beamforming and massive MIMO can improve signal strength and reliability in complex factory environments. Wi-Fi 7, on the other hand, offers high throughput using wider channels (up to 320 MHz) and 4096-QAM modulation, making it well-suited for transmitting high-bandwidth data such as video. Features like multi-link and multi-AP operation further enhance connection stability as AMRs move across coverage zones. Although both technologies provide significant benefits, several challenges still need to be addressed. These include optimal base station placement for private 5G networks, compatibility and interference with existing systems for Wi-Fi 7, and the need for dynamic network management to ensure quality of service (QoS) in constantly changing environments. For a more detailed discussion, we refer the reader to [139].

5.2.2. Cloud computing and data processing

Because perception and decision-making are offloaded to an on-premise cloud, the system must support real-time data processing, scalability, and fault tolerance. One major challenge is dynamic resource allocation, as compute demands vary depending on robot location and task complexity. In infrastructure-sparse areas, AMRs must rely on onboard sensing and computation. In contrast, infrastructure-rich zones can produce high volumes of data from multiple sources, such as ceiling-mounted cameras and onboard sensors, which may cause temporary spikes in processing requirements. A critical design question concerns which processing tasks should be offloaded to the cloud and how competing requests should be prioritized [140].

By allocating computational resources efficiently, the footprint of the on-premise cloud can be minimized, which is an important consideration when scaling the system to hundreds of robots and infrastructure devices. However, achieving this efficiency poses significant challenges, particularly in maintaining real-time performance under fluctuating loads. Moreover, the on-premise cloud service must be robust to node failures, software errors, and hardware degradation, which requires redundancy, distributed processing capabilities, and health monitoring mechanisms. Recent work has demonstrated how dynamic resource allocation can be combined with delay-aware control to maintain real-time performance in edge-offloaded robotic systems by adapting computational resources and controller complexity to changing communication and processing conditions [141]. Extending such approaches to infrastructure-based AMR systems that integrate perception, planning, and fleet coordination represents an important direction for future research.

5.3. System-level considerations

5.3.1. System integration

System integration introduces significant challenges across hardware and software. On the hardware side, sensor types and placements must be carefully planned, both onboard the robots and in the environment, to ensure adequate coverage and accuracy for localization and perception tasks. Network design is equally important for maintaining low latency and high reliability, particularly in large or frequently changing environments. Factory layouts that evolve over time, for example through the relocation of machinery or storage areas, may require reconfiguration of sensors and networks to preserve system performance. These dynamics introduce significant maintenance challenges.

The hardware design directly shape system-level characteristics like sensing coverage, latency, communication jitter, and computational availability. These characteristics, in turn, place constraints on the software stack. For example, limited sensor visibility or intermittent network delays must be accounted for in perception algorithms, which may need to operate with incomplete or delayed data. Similarly, decision-making components must be designed to tolerate fluctuations in response time and handle transitions between centralized and local execution. Effectively, software modules must be designed to compensate for hardware and network limitations, which adds complexity to their design and integration.

Although often treated as implementation details, these cross-cutting concerns profoundly affect system performance, reliability, and scalability. The intricate dependencies between hardware, software, and communication layers can lead to emergent behaviors that are difficult to predict and diagnose. This complexity highlights the need for principled, systems-level approaches to AMR integration that go beyond modular software design and consider the full stack as a unified whole.

Current interoperability standards, such as VDA 5050 [33], standardize communication between fleet management systems and mobile robots, but leave the realization of localization, perception, and navigation to the robot implementation. In contrast, RAIL explicitly considers how these functions may be distributed across robots, infrastructure, and cloud resources through a shared state. As infrastructure-based AMR systems become increasingly adopted, future interoperability standards should therefore also consider mechanisms for exchanging shared environment representations (e.g., occupancy maps, tracked objects, and semantic scene information), infrastructure-derived perception, and other elements of the shared state between robots and infrastructure services. Such extensions would facilitate vendor-independent deployment of infrastructure-based AMR systems while preserving the interoperability objectives of existing standards.

5.3.2. Safety assurance

Because robots operate alongside unprotected humans and manually driven vehicles, safety assurance is essential. Deep learning (DL) models, which are the core of the system intelligence, are particularly opaque in nature. They often consists of huge neural networks with millions of parameters. Consequently, there are very limited options to analyze misbehavior from a functional safety perspective, as neither traditional code review nor safety assurance practices are applicable. Ongoing research is focused on developing methodologies that enable the safe development and integration of DL-based components into safety-critical systems. The latest developments in the safety assurance guidelines for DL-based systems include AMLAS [142] and, most recently, AI-FSM [143], together with experimental compliance practices [144,145] and relevant standards [146,147]. Although these frameworks offer guidance for adopting ML in safety-critical applications, significant challenges remain in building fully compliant complex systems, including deriving allocated safety metrics to individual DL components.

5.3.3. Testing

Manual testing remains common in industry, but verifying autonomous systems with machine learning-based perception and uncertain actor dynamics poses major challenges. Scenario-based verification is increasingly needed to support safety arguments. Tools like scenario-based generation [148] help generate diverse, realistic scenarios, while falsification techniques [149,150] enables search-based falsification against temporal logic specifications. These methods increase demands on the simulation environment, which must accurately model both the physical dynamics and the perception system, including sensor noise and classification errors.

5.3.4. Security and privacy

Infrastructure-based AMR systems face significant security challenges, particularly in wireless communication and control system resilience. Wireless communication exposes the system to threats such as jamming, spoofing, and man-in-the-middle attacks [151]. Effective protection requires both proactive measures like secure authentication and reactive strategies such as real-time network anomaly detection. Additionally, AI-based control and perception systems are vulnerable to adversarial inputs that can cause unsafe behavior [152], emphasizing the need for robust models and runtime safeguards.

Privacy risks arise from visual sensors that may capture sensitive data, including images of workers, proprietary products, or confidential processes. Storing such data for AI training raises legal and ethical concerns, particularly under regulations like the GDPR. An emerging solution is federated learning [153], where AI models are trained locally at each deployment site using only local data. Instead of sharing raw data, only model updates are sent to a central server. This method helps protect privacy by keeping sensitive data within local boundaries while still enabling system-wide improvements.

5.4. Human-centered design

5.4.1. Human-robot interaction

The user experience evaluation revealed several challenges in human–robot interaction within shared industrial environments, particularly regarding collaboration, communication, and user acceptance. Forklift drivers, in particular, reported a high mental workload when interacting with the AMRs because they found the robots difficult to spot and their behavior occasionally unpredictable. Identifying the specific causes of these negative experiences and determining how to mitigate them remains a complex and open-ended problem. In fact, it aligns with the notion of a wicked problem [154], where the problem definition itself is fluid and solutions are difficult to evaluate definitively.

One promising way to reduce cognitive strain is to establish clear traffic rules and interaction protocols. These reduce ambiguity, minimize ad hoc decision-making, and help human workers better anticipate robot behavior. However, designing rules that ensure both safety and efficiency while allowing flexibility is challenging. In practice, strict adherence to predefined rules is not always feasible. Unexpected obstacles, robot malfunctions, or deviations from normal operations may require temporary deviations from standard behavior.

In the deployment at Volvo, humans could manually take control of the robots to handle unexpected situations. While effective, this interrupts workflow and increases the workload for the humans, potentially hindering acceptance of the system. Improving collaboration between humans and robots is therefore essential. Robots must be able to infer human intentions through cues such as gestures, gaze, or context, while clearly communicating their own intentions in ways that are intuitive for non-experts. Beyond predictable motion, tools like LED indicators, sounds, or display screens can improve transparency and reduce uncertainty.

If not carefully designed, however, systems that require users to interpret multiple signals or give nuanced input to the system may inadvertently increase cognitive load further. A human-centered design approach must therefore strike a careful balance: enabling expressive, intuitive communication to enable seamless collaboration between human and robots, while preserving simplicity and minimizing the user's need to learn or adapt to overly complex robot behavior.

6. Conclusions

To address the fragmented landscape of infrastructure-based AMR systems, this paper presents a modular reference architecture that combines infrastructure-based sensing, on-premise cloud computing, and onboard autonomy. We reviewed enabling technologies for localization, perception, and planning; and further demonstrated the practical feasibility of the approach through a real-world industrial deployment. Based on the review and industrial evaluation, we identified challenges in several research areas that warrant further investigation. As robotics moves beyond isolated automation solutions toward more collaborative, scalable, and adaptive systems, we argue that infrastructure-enabled autonomy will play a central role. We hope this paper provides a foundation for both academic research and industrial adoption of the next generation of AMR systems.

CRediT authorship contribution statement

Erik Brorsson: Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Kristian Ceder:** Writing – original draft. **Ze Zhang:** Writing – original draft. **Sabino Francesco Roselli:** Writing – original draft. **Endre Erős:** Writing – original draft. **Martin Dahl:** Writing – original draft, Software, Methodology, Conceptualization. **Beatrice Alenljung:** Writing – original draft. **Jessica Lindblom:** Writing – original draft. **Thanh Bui:** Writing – original draft. **Atieh Hanna:** Conceptualization. **Emmanuel Dean:** Software, Methodology, Conceptualization. **Lennart Svensson:** Supervision. **Kristofer Bengtsson:** Supervision, Software, Methodology, Conceptualization. **Per-Lage Götvall:** Supervision, Methodology, Conceptualization. **Knut Åkesson:** Writing – original draft, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by AB Volvo, the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, and the Vinnova funded projects SMILE IV (2023-00789) and AIHURO (2022-03012).

Data availability

No data was used for the research described in the article.

References

[1] G. Fragapane, et al., Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda, European J. Oper. Res. 294 (2) (2021) 405–426.
[2] Í.R. da Costa Barros, T.P. Nascimento, Robotic mobile fulfillment systems: A survey on recent developments and research opportunities, Robot. Auton. Syst. 137 (2021) 103729.
[3] R. Bernardo, et al., Survey on robotic systems for internal logistics, J. Manuf. Syst. 65 (2022) 339–350.
[4] T. Lackner, et al., Review of autonomous mobile robots in intralogistics: state-of-the-art, limitations and research gaps, Procedia CIRP 130 (2024) 930–935.
[5] J. Fottner, et al., Autonomous systems in intralogistics: state of the art and future research challenges, Logist. Res. 14 (1) (2021) 1–41.
[6] M.A. Niloy, et al., Critical design and control issues of indoor autonomous mobile robots: A review, IEEE Access 9 (2021) 35338–35370.
[7] F. Rubio, et al., A review of mobile robots: Concepts, methods, theoretical framework, and applications, Int. J. Adv. Robot. Syst. 16 (2) (2019) 1729881419839596.
[8] M.B. Alatise, G.P. Hancke, A review on challenges of autonomous mobile robot and sensor fusion methods, IEEE Access 8 (2020) 39830–39846.
[9] G. Hu, et al., Cloud robotics: architecture, challenges and applications, IEEE Netw. 26 (3) (2012) 21–28.
[10] P. Simoens, et al., The internet of robotic things: A review of concept, added value and applications, Int. J. Adv. Robot. Syst. 15 (1) (2018) 1729881418759424.
[11] R.C. Mello, et al., The PoundCloud framework for ROS-based cloud robotics: Case studies on autonomous navigation and human–robot interaction, Robot. Auton. Syst. 150 (2022) 103981.
[12] M. Balogh, et al., Cloud-controlled autonomous mobile robot platform, in: IEEE 32nd Annu. Int. Symp. Pers., Indoor Mobile Radio Commun., IEEE, 2021, pp. 1–6.
[13] E. Masehian, et al., Cooperative mapping of unknown environments by multiple heterogeneous mobile robots with limited sensing, Robot. Auton. Syst. 87 (2017) 188–218.
[14] R. Arumugam, et al., DAvinCi: A cloud computing framework for service robots, in: IEEE Int. Conf. Robot. Autom., IEEE, 2010, pp. 3084–3089.
[15] L. Riazuelo, et al., C2tam: A cloud framework for cooperative tracking and mapping, Robot. Auton. Syst. 62 (4) (2014) 401–413.
[16] M. Karrer, et al., CVI-SLAM—collaborative visual-inertial SLAM, IEEE Robot. Autom. Lett. 3 (4) (2018) 2762–2769.
[17] G. Mohanarajah, et al., Cloud-based collaborative 3D mapping in real-time with low-cost robots, IEEE Trans. Autom. Sci. Eng. 12 (2) (2015) 423–431.
[18] A.T. Rashid, et al., Multi-robot localization and orientation estimation using robotic cluster matching algorithm, Robot. Auton. Syst. 63 (2015) 108–121.
[19] M.S. Alam, et al., Fiducial markers and particle filter based localization and navigation framework for an autonomous mobile robot, SN Comput. Sci. 5 (6) (2024) 748.

[20] X. Zhang, et al., An autonomous robotic system for intralogistics assisted by distributed smart camera network for navigation, in: IEEE Int. Conf. Autom. Sci. Eng., IEEE, 2018, pp. 1224–1229.

[21] J.-H. Lee, H. Hashimoto, Controlling mobile robots in distributed intelligent sensor network, IEEE Trans. Ind. Electron. 50 (5) (2003) 890–902.

[22] A. Reina, et al., zePPeLIN: Distributed path planning using an overhead camera network, Int. J. Adv. Robot. Syst. 11 (8) (2014) 119.

[23] F.-J. Streit, et al., Vision-based path construction and maintenance for indoor guidance of autonomous ground vehicles based on collaborative smart cameras, in: Int. Conf. Distrib. Smart Cameras, 2016, pp. 44–49.

[24] J. Zhang, et al., Automated guided vehicles and autonomous mobile robots for recognition and tracking in civil engineering, Autom. Constr. 146 (2023) 104699.

[25] C. Badue, et al., Self-driving cars: A survey, Expert Syst. Appl. 165 (2021) 113816.

[26] E. Yurtsever, et al., A survey of autonomous driving: Common practices and emerging technologies, IEEE Access 8 (2020) 58443–58469.

[27] J. Zhao, et al., Autonomous driving system: A comprehensive survey, Expert Syst. Appl. 242 (2024) 122836.

[28] S. Hakak, et al., Autonomous vehicles in 5G and beyond: A survey, Veh. Commun. 39 (2023) 100551.

[29] A. Loganathan, N.S. Ahmad, A systematic review on recent advances in autonomous mobile robot navigation, Eng. Sci. Technol. an Int. J. 40 (2023) 101343.

[30] B. Kehoe, et al., A survey of research on cloud robotics and automation, IEEE Trans. Autom. Sci. Eng. 12 (2) (2015) 398–409.

[31] S. Macenski, et al., The marathon 2: A navigation system, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2020, pp. 2718–2725.

[32] R. Siegwart, et al., Introduction to Autonomous Mobile Robots, MIT Press, 2011.

[33] Verband der Automobilindustrie (VDA), VDMA, VDA 5050: Communication interface for automated guided vehicle systems (AGV), version 3.0.0, 2025, https://www.vda.de/en/topics/automotive-industry/vda-5050. Official specification, (Accessed 16 July 2026).

[34] J. Huang, et al., Indoor positioning systems of mobile robots: A review, Robotics 12 (2) (2023) 47.

[35] C. Cadena, et al., Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age, IEEE Trans. Robot. 32 (6) (2017) 1309–1332.

[36] Q. Zou, et al., A comparative analysis of LiDAR SLAM-based indoor navigation for autonomous vehicles, IEEE Trans. Intell. Transp. Syst. 23 (7) (2021) 6907–6921.

[37] A. Macario Barros, et al., A comprehensive survey of visual slam algorithms, Robotics 11 (1) (2022) 24.

[38] S. Macenski, et al., From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2, Robot. Auton. Syst. (2023).

[39] S. Macenski, I. Jambrecic, SLAM toolbox: SLAM for the dynamic world, J. Open Source Softw. 6 (61) (2021) 2783.

[40] G. Grisetti, et al., Improved techniques for grid mapping with rao-blackwellized particle filters, IEEE Trans. Robot. 23 (1) (2007) 34–46.

[41] K. Konolige, et al., Efficient sparse pose adjustment for 2D mapping, in: IEEE/RSJ Int. Conf. Intell. Robots Syst., IEEE, 2010, pp. 22–29.

[42] W. Hess, et al., Real-time loop closure in 2D LIDAR SLAM, in: IEEE Int. Conf. Robot. Autom., IEEE, 2016, pp. 1271–1278.

[43] C. Campos, et al., Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam, IEEE Trans. Robot. 37 (6) (2021) 1874–1890.

[44] P.K. Panigrahi, S.K. Bisoy, Localization strategies for autonomous mobile robots: A review, J. King Saud Univ.–Comput. Inf. Sci. 34 (8) (2022) 6019–6039.

[45] K.-T. Song, Y.C. Chang, Design and implementation of a pose estimation system based on visual fiducial features and multiple cameras, in: Int. Autom. Control Conf., IEEE, 2018, pp. 1–6.

[46] L. Liu, et al., Computing systems for autonomous driving: State of the art and challenges, IEEE Internet Things J. 8 (8) (2020) 6469–6486.

[47] J. Philion, S. Fidler, Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d, in: Eur. Conf. Comput. Vis., Springer, 2020, pp. 194–210.

[48] A.H. Lang, et al., Pointpillars: Fast encoders for object detection from point clouds, in: IEEE Conf. Comput. Vis. Pattern Recog., 2019, pp. 12697–12705.

[49] Z. Li, et al., Bevformer: learning bird's-eye-view representation from lidar-camera via spatiotemporal transformers, IEEE Trans. Pattern Anal. Mach. Intell. (2024).

[50] T. Liang, et al., Bevfusion: A simple and robust lidar-camera fusion framework, Adv. Neural Inform. Process. Syst. 35 (2022) 10421–10434.

[51] H. Li, et al., Delving into the devils of bird's-eye-view perception: A review, evaluation and recipe, IEEE Trans. Pattern Anal. Mach. Intell. 46 (4) (2023) 2151–2170.

[52] F. Fleuret, et al., Multicamera people tracking with a probabilistic occupancy map, IEEE Trans. Pattern Anal. Mach. Intell. 30 (2) (2008) 267–282.

[53] G. Roig, et al., Conditional random fields for multi-camera object detection, in: IEEE/CVF Int. Conf. Comput. Vis., IEEE, 2011, pp. 563–570.

[54] Y. Hou, et al., Multiview detection with feature perspective transformation, in: Eur. Conf. Comput. Vis., Springer, 2020, pp. 1–18.

[55] S. Liu, et al., Towards vehicle-to-everything autonomous driving: A survey on collaborative perception, 2023, arXiv:2308.16714.

[56] Y. Hu, et al., Collaboration helps camera overtake lidar in 3d detection, in: IEEE Conf. Comput. Vis. Pattern Recog., 2023, pp. 9243–9252.

[57] Y. Li, et al., Multi-robot scene completion: Towards task-agnostic collaborative perception, in: K. Liu, D. Kulic, J. Ichnowski (Eds.), Conf. Robot Learn, in: Proceedings of Machine Learning Research, vol. 205, PMLR, 2023, pp. 2062–2072.

[58] Y. Li, et al., Learning distilled collaboration graph for multi-agent perception, 2022, arXiv:2111.00643.

[59] R. Song, et al., Collaborative semantic occupancy prediction with hybrid feature fusion in connected automated vehicles, in: IEEE Conf. Comput. Vis. Pattern Recog., 2024, pp. 17996–18006.

[60] R. Xu, et al., V2x-vit: Vehicle-to-everything cooperative perception with vision transformer, in: Eur. Conf. Comput. Vis., Springer, 2022, pp. 107–124.

[61] X. Weng, et al., 3D multi-object tracking: A baseline and new evaluation metrics, in: IEEE/RSJ Int. Conf. Intell. Robots Syst., IEEE, 2020, pp. 10359–10366.

[62] H. kuang Chiu, et al., Probabilistic 3D multi-object tracking for autonomous driving, 2020, arXiv:2001.05673.

[63] T. Yin, et al., Center-based 3d object detection and tracking, in: IEEE Conf. Comput. Vis. Pattern Recog., 2021, pp. 11784–11793.

[64] A. Kim, et al., Eagermot: 3d multi-object tracking via sensor fusion, in: IEEE Int. Conf. Robot. Autom., IEEE, 2021, pp. 11315–11321.

[65] J.L. Williams, Marginal multi-Bernoulli filters: RFS derivation of MHT, JIPDA, and association-based MeMBer, IEEE Trans. Aerosp. Electron. Syst. 51 (3) (2015) 1664–1687.

[66] Á.F. García-Fernández, et al., Poisson multi-Bernoulli mixture filter: Direct derivation and implementation, IEEE Trans. Aerosp. Electron. Syst. 54 (4) (2018) 1883–1901.

[67] T. Teepe, et al., EarlyBird: early-fusion for multi-view tracking in the bird's eye view, in: IEEE/CVF Winter Conf. Appl. Comput. Vis., 2024, pp. 102–111.

[68] H.-k. Chiu, et al., Probabilistic 3D multi-modal, multi-object tracking for autonomous driving, in: IEEE Int. Conf. Robot. Autom., IEEE, 2021, pp. 14227–14233.

[69] Y. Xu, et al., Multi-view people tracking via hierarchical trajectory composition, in: IEEE Conf. Comput. Vis. Pattern Recog., 2016, pp. 4256–4265.

[70] C.-C. Cheng, et al., Rest: A reconfigurable spatial-temporal graph model for multi-camera multi-object tracking, in: IEEE/CVF Int. Conf. Comput. Vis., 2023, pp. 10051–10060.

[71] J. Ong, et al., A Bayesian filter for multi-view 3D multi-object tracking with occlusion handling, IEEE Trans. Pattern Anal. Mach. Intell. 44 (5) (2020) 2246–2263.

[72] X. Zhou, et al., Tracking objects as points, in: Eur. Conf. Comput. Vis., Springer, 2020, pp. 474–490.

[73] C. Zhang, et al., Motiontrack: End-to-end transformer-based multi-object tracking with lidar-camera fusion, in: IEEE Conf. Comput. Vis. Pattern Recog., 2023, pp. 151–160.

[74] T. Meinhardt, et al., Trackformer: Multi-object tracking with transformers, in: IEEE Conf. Comput. Vis. Pattern Recog., 2022, pp. 8844–8854.

[75] H.-K. Chiu, et al., Probabilistic 3d multi-object cooperative tracking for autonomous driving via differentiable multi-sensor Kalman filter, in: IEEE Int. Conf. Robot. Autom., IEEE, 2024, pp. 18458–18464.

[76] T. Hu, et al., Integrated detection and tracking framework for 3D multi-object tracking in vehicle-infrastructure cooperation, Int. J. Adv. Comput. Sci. Appl. 15 (11) (2024).

[77] C. Schöller, et al., What the constant velocity model can teach us about pedestrian motion prediction, IEEE Robot. Autom. Lett. 5 (2) (2020) 1696–1703.

[78] D. Helbing, P. Molnar, Social force model for pedestrian dynamics, Phys. Rev. E 51 (5) (1995) 4282.

[79] J. van den Berg, et al., Reciprocal velocity obstacles for real-time multi-agent navigation, in: IEEE Int. Conf. Robot. Autom., 2008, pp. 1928–1935.

[80] A. Gupta, et al., Social GAN: Socially acceptable trajectories with generative adversarial networks, in: IEEE Conf. Comput. Vis. Pattern Recog., 2018, pp. 2255–2264.

[81] O. Makansi, et al., Overcoming limitations of mixture density networks: A sampling and fitting framework for multimodal future prediction, in: IEEE Conf. Comput. Vis. Pattern Recog., 2019.

[82] K. Mangalam, et al., From goals, waypoints & paths to long term human trajectory forecasting, in: IEEE/CVF Int. Conf. Comput. Vis., 2021, pp. 15213–15222.

[83] Z. Zhang, et al., Future-oriented navigation: Dynamic obstacle avoidance with one-shot energy-based multimodal motion prediction, IEEE Robot. Autom. Lett. 10 (8) (2025) 8043–8050.

[84] L. Fang, et al., TPNet: Trajectory proposal network for motion prediction, in: IEEE Conf. Comput. Vis. Pattern Recog., 2020, pp. 6797–6806.
[85] T. Salzmann, et al., Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data, in: Eur. Conf. Comput. Vis., Springer, 2020, pp. 683–700.
[86] C. Yang, et al., Social self-attention generative adversarial networks for human trajectory prediction, IEEE Trans. Artif. Intell. 5 (4) (2024) 1805–1815.
[87] K. Guo, et al., End-to-end trajectory distribution prediction based on occupancy grid maps, in: IEEE Conf. Comput. Vis. Pattern Recog., 2022, pp. 2232–2241.
[88] B. Paden, et al., A survey of motion planning and control techniques for self-driving urban vehicles, IEEE Trans. Intell. Veh. 1 (1) (2016) 33–55.
[89] S. Teng, et al., Motion planning for autonomous driving: The state of the art and future perspectives, IEEE Trans. Intell. Veh. 8 (6) (2023) 3692–3711.
[90] G.D. Konstantakopoulos, et al., Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification, Oper. Res. 22 (3) (2022) 2033–2062.
[91] S. Lin, et al., A review of path-planning approaches for multiple mobile robots, Machines 10 (9) (2022) 773.
[92] S. Jun, et al., Scheduling of autonomous mobile robots with conflict-free routes utilising contextual-bandit-based local search, Int. J. Prod. Res. 60 (13) (2022) 4090–4116.
[93] S.F. Roselli, et al., Solving the conflict-free electric vehicle routing problem using SMT solvers, in: Mediterr. Conf. Control Autom., IEEE, 2021, pp. 542–547.
[94] S.F. Roselli, et al., A compositional algorithm for the conflict-free electric vehicle routing problem, IEEE Trans. Autom. Sci. Eng. 19 (3) (2022) 1405–1421.
[95] F. Popolizio, et al., Online conflict-free scheduling of fleets of autonomous mobile robots, in: IEEE Int. Conf. Autom. Sci. Eng., IEEE, 2024, pp. 3063–3068.
[96] A. Bogyrbayeva, et al., Machine learning to solve vehicle routing problems: A survey, IEEE Trans. Intell. Transp. Syst. 25 (6) (2024) 4754–4772.
[97] M.A.L. Silva, et al., A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems, Expert Syst. Appl. 131 (2019) 148–171.
[98] K. Zhang, et al., Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach, Transp. Res. Part C: Emerg. Technol. 121 (2020) 102861.
[99] R. Bai, et al., Analytics and machine learning in vehicle routing research, Int. J. Prod. Res. 61 (1) (2023) 4–30.
[100] M. Ghallab, et al., Automated Planning: Theory and Practice, Elsevier, 2004.
[101] C. Aeronautiques, et al., Pddl| the Planning Domain Definition Language, Technical Report, Tech. Rep., 1998.
[102] M. Fox, D. Long, PDDL2. 1: An extension to PDDL for expressing temporal planning domains, J. Artif. Intell. Res. 20 (2003) 61–124.
[103] N.W. Paton, O. Diaz, Active database systems, ACM Comput. Surv. 31 (1) (1999) 63–103.
[104] M. Sköldstam, et al., Supervisory Control Applied to Automata Extended with Variables-Revised, Relatório Técnico, Chalmers University of Technology, Goteborg, 2008.
[105] S.J. Russell, P. Norvig, Artificial Intelligence: a Modern Approach, third ed., pearson, 2016.
[106] R.E. Bryant, Graph-based algorithms for Boolean function manipulation, IEEE Trans. Comput. 100 (8) (1986) 677–691.
[107] H.A. Kautz, et al., Planning as satisfiability, in: ECAI, Vol. 92, Citeseer, 1992, pp. 359–363.
[108] A. Pnueli, The temporal logic of programs, in: IEEE Annu. Symp. Found. Comput. Sci., ieee, 1977, pp. 46–57.
[109] C. Baier, J.-P. Katoen, Principles of Model Checking, MIT Press, 2008.
[110] E. Amir, A. Chang, Learning partially observable deterministic action models, J. Artif. Intell. Res. 33 (2008) 349–402.
[111] A. Farooqui, et al., On active learning for supervisor synthesis, IEEE Trans. Autom. Sci. Eng. 21 (1) (2024) 78–90.
[112] S. Alarabi, M. Santora, Review:Path planning techniques for automated guided vehicles (AGVs), in: Proc. 9th Asia-Pacific Conf. Intell. Robot Syst., 2024, pp. 33–38.
[113] Y. Zhou, et al., A survey on the key technologies of UAV motion planning, Drones 9 (3) (2025).
[114] J. Guo, et al., An improvement of D* algorithm for mobile robot path planning in partial unknown environment, in: Int. Conf. Intell. Comput. Technol. Autom., Vol. 3, 2009, pp. 394–397.
[115] K. Karur, et al., A survey of path planning algorithms for mobile robots, Vehicles 3 (3) (2021) 448–468.
[116] L. Dong, et al., A review of mobile robot motion planning methods: from classical motion planning workflows to reinforcement learning-based architectures, J. Syst. Eng. Electron. 34 (2) (2023) 439–459.
[117] C. Souza de Oliveira, et al., Trajectory planning for autonomous cars in low-structured and unstructured environments: A systematic review, IEEE Access 13 (2025) 48841–48871.
[118] S. Hwang, et al., Safe motion planning and control for mobile robots: A survey, Int. J. Control. Autom. Syst. 22 (2024) 2955–2969.
[119] J. Wu, et al., An improved timed elastic band (TEB) algorithm of autonomous ground vehicle (AGV) in complex environment, Sensors 21 (24) (2021).
[120] X. Xiao, et al., Motion planning and control for mobile robot navigation using machine learning: a survey, Auton. Robots 46 (5) (2022) 569–597.
[121] K. Ceder, et al., Bird's-eye-view trajectory planning of multiple robots using continuous deep reinforcement learning and model predictive control, in: IEEE/RSJ Int. Conf. Intell. Robots Syst., 2024, pp. 8002–8008.
[122] Z. Zhang, et al., Collision-free trajectory planning of mobile robots by integrating deep reinforcement learning and model predictive control, in: IEEE Int. Conf. Autom. Sci. Eng., 2023, pp. 1–7.
[123] U. Patel, et al., DWA-RL: Dynamically feasible deep reinforcement learning policy for robot navigation among mobile obstacles, in: IEEE Int. Conf. Robot. Autom., 2021, pp. 6057–6063.
[124] S. Sadraddini, C. Belta, Robust temporal logic model predictive control, in: Annu. Allerton Conf. Commun. Control Comput., 2015, pp. 772–779.
[125] M. Charitidou, D.V. Dimarogonas, Distributed MPC with continuous-time STL constraint satisfaction guarantees, IEEE Control. Syst. Lett. 8 (2024) 211–216.
[126] L. Lindemann, D.V. Dimarogonas, Control barrier functions for signal temporal logic tasks, IEEE Control. Syst. Lett. 3 (1) (2019) 96–101.
[127] Y. Guo, et al., A survey of trajectory planning methods for autonomous driving—Part I: Unstructured scenarios, IEEE Trans. Intell. Veh. 9 (9) (2024) 5407–5434.
[128] S. Garrido-Jurado, et al., Generation of fiducial marker dictionaries using mixed integer linear programming, Pattern Recognit. 51 (2016) 481–491.
[129] H. Ahmed, et al., Realistisk träningsdata med 3D Gaussian splatting: Bygga digitala tvillingar för generering av ground-truth-data i unreal engine 5, 2025.
[130] Z. Zhang, A flexible new technique for camera calibration, IEEE Trans. Pattern Anal. Mach. Intell. 22 (11) (2000) 1330–1334.
[131] Rerun Development Team, Rerun: A visualization SDK for multimodal data, 2024, Available from https://www.rerun.io/ and https://github.com/rerun-io/rerun.
[132] F.J. Romero-Ramirez, et al., Speeded up detection of squared fiducial markers, Image Vis. Comput. 76 (2018) 38–47.
[133] B. Alenljung, et al., Towards a framework of human-robot interaction strategies from an operator 5.0 perspective, in: Adv. Manuf. Technol. XXXVI, IOS Press, 2023, pp. 81–86.
[134] J. Lindblom, B. Alenljung, The ANEMONE: theoretical foundations for UX evaluation of action and intention recognition in human-robot interaction, Sensors 20 (15) (2020) 4284.
[135] B. Alenljung, J. Lindblom, Introducing mobile robots on the shop floor: User experience issues, in: Int. Conf. Hum.-Comput. Interact., Springer, 2025, pp. 3–19.
[136] User Experience Questionnaire (UEQ), UEQ - User Experience Questionnaire, 2024, https://www.ueq-online.org/. (Accessed 1 July 2025).
[137] G. Hess, et al., Splatad: Real-time lidar and camera rendering with 3d Gaussian splatting for autonomous driving, in: IEEE Conf. Comput. Vis. Pattern Recog., 2025, pp. 11982–11992.
[138] A. Tonderski, et al., Neurad: Neural rendering for autonomous driving, in: IEEE Conf. Comput. Vis. Pattern Recog., 2024, pp. 14895–14904.
[139] J. John, et al., Industry 4.0 and beyond: The role of 5G, WiFi 7, and time-sensitive networking (TSN) in enabling smart manufacturing, Future Internet 16 (9) (2024) 345.
[140] O. Saha, P. Dasgupta, A comprehensive survey of recent trends in cloud robotics architectures and applications, Robotics 7 (3) (2018) 47.
[141] A.S. Seisa, et al., Optimization of edge-offloading for centralized controllers through dynamic computational resource allocation, IEEE Internet Things J. (2026).
[142] R. Hawkins, et al., Guidance on the Assurance of Machine Learning in Autonomous Systems (AMLAS), 2021, arXiv:2102.01564.
[143] SAFEXPLAIN, D2.1: Safexplain safety lifecycle considerations, 2024, https://safexplain.eu/deliverables.
[144] M. Borg, et al., Ergo, SMIRK is safe: a safety case for a machine learning component in a pedestrian automatic emergency brake system, Softw. Qual. J. (2023).
[145] SAFEXPLAIN, D3.3: Final proofs-of-concept, arguments, and DL components and libraries, 2025, https://safexplain.eu/deliverables.
[146] International Organization for Standardization, Road vehicles — Safety of the intended functionality, 2022, (ISO Standard No. 21448:2022).
[147] International Organization for Standardization, Artificial Intelligence — Functional Safety and AI Systems (ISO Standard No. 5469:2024), Technical Report, ISO, 2024.
[148] D.J. Fremont, et al., Scenic: a language for scenario specification and scene generation, in: ACM SIGPLAN Conf. Program. Lang. Des. Implement., in: PLDI 2019, Association for Computing Machinery, 2019, pp. 63–78.
[149] A. Donzé, Breach, a toolbox for verification and parameter synthesis of hybrid systems, in: Int. Conf. Comput. Aided Verif., in: Lecture Notes in Computer Science, vol. 6174, Springer, 2010, pp. 167–170.
[150] T. Dreossi, et al., VerifAI: a toolkit for the formal design and analysis of artificial intelligence-based systems, in: Int. Conf. Comput. Aided Verif., 2019.

[151] R. Sedar, et al., A comprehensive survey of V2X cybersecurity mechanisms and future research paths, IEEE Open J. Commun. Soc. 4 (2023) 325–391.
[152] M. Girdhar, et al., Cybersecurity of autonomous vehicles: A systematic literature review of adversarial attacks and defense models, IEEE Open J. Veh. Technol. 4 (2023) 417–437.
[153] C. Zhang, et al., A survey on federated learning, Knowl.-Based Syst. 216 (2021) 106775.
[154] H.W. Rittel, M.M. Webber, Dilemmas in a general theory of planning, Policy Sci. 4 (2) (1973) 155–169.